

Project Proposal: Doomagotchi

David Golding 200238

Open Window, School of Fundamentals

Interactive Development 300

Tsungai Katsuri

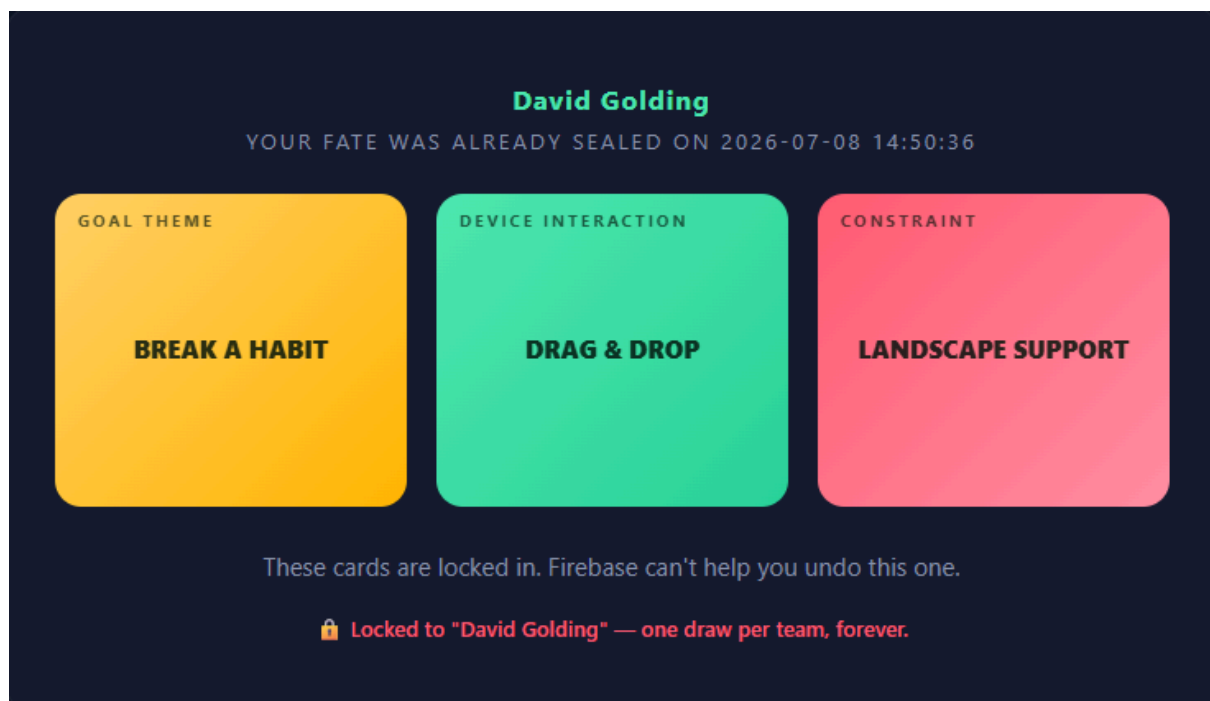
A) Project Title

Doomagotchi

Your pet feels every scroll.

B) Conceptualisation and Problem Statement

Inspiration Cards:



These three cards are fixed. The app helps users break the doomscrolling habit. Drag & Drop is used to feed the pet (food and medicine give a small health boost). Every screen is designed to work properly in landscape orientation.

The Problem:

Young adults (especially students) lose hours every day to mindless scrolling on Instagram, TikTok and similar apps. They know it hurts focus, sleep and mood, yet traditional blockers and pure willpower almost always fail.

Who it is for and why they care:

Primary audience: Gen Z and younger Millennials (18–32), university students and early-career people who already feel the damage of doomscrolling. A living pixel pet that visibly suffers when they scroll creates emotional accountability that a simple timer cannot match.

Core concept: Doomagotchi is a pixel Tamagotchi-style pet whose health is directly tied to the time you spend doomscrolling. The longer you scroll, the sicker the pet becomes. The longer you stay off those apps, the healthier it gets. You can also drag food or medicine onto the pet for a small health boost.

C) Core Features

SMART Objectives

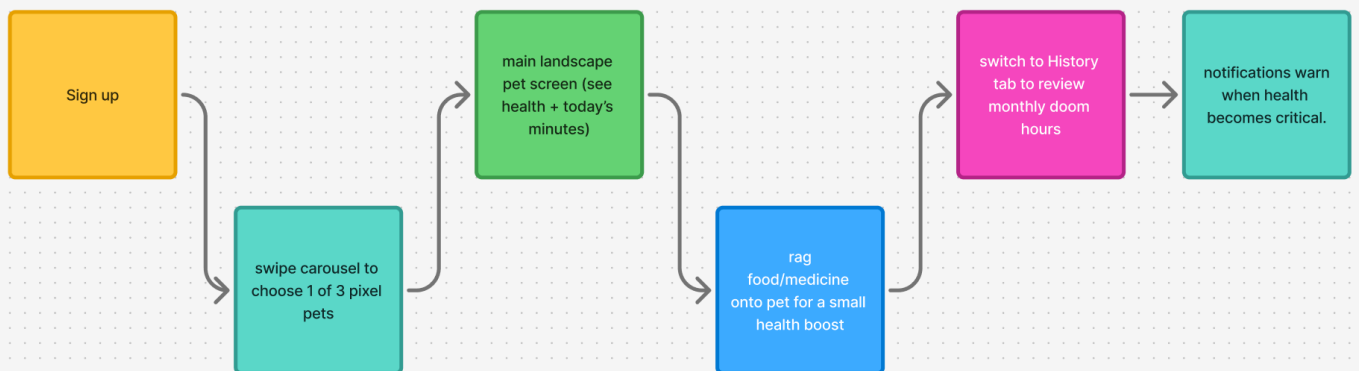
- **Specific:** Reduce daily time spent on selected social apps by making the consequence (sick pet) immediately visible.
- **Measurable:** Track minutes of doomscrolling per day and pet health score (0–100).
- **Achievable:** Uses Android UsageStatsManager + simple drag interactions.
- **Relevant:** Directly targets the most common digital habit in the target market.
- **Time-bound:** Pet health reacts within hours; clear monthly history available.

Feature Prioritisation		
Priority	Feature	Description
MVP	Account & Pet Select	Firestore Auth. At first launch the user swipes through a carousel of 3 pixel pets and picks one
MVP	Pet Health Engine	Health (0–100) decays with doomscroll minutes tracked via Android UsageStatsManager. Recovers during clean time. Visual states: Happy → Tired → Sick → Critical.
MVP	Drag & Drop Care	User drags food or medicine onto the pet. Each successful drop gives a small

Feature Prioritisation

Priority	Feature	Description
		health boost. This is the main use of the Drag & Drop card.
MVP	History / Calendar	Second screen showing daily doomscrolling hours for the current month so the user can see time actually spent and saved.
MVP	Landscape UI	All screens designed and tested to work properly in landscape orientation (constraint card).
Nice-to-Have	Health Notifications	Local notifications when pet health drops into Sick or Critical range.
Nice-to-Have	Light Entertainment	Simple idle animations and particle effects when the pet is fed. No extra mini-games.
Future	More pets / cosmetics	Additional sprites or small customisation options later.

Simple User Flow:



Sign up → swipe carousel to choose 1 of 3 pixel pets → main landscape pet screen (see health + today's minutes) → drag food/medicine onto pet for a small health boost → switch to History tab to review monthly doom hours → notifications warn when health becomes critical.

D) Technical Planning

Stack & Libraries

- **Frontend:** React Native (Expo managed workflow) — primary testing on Android
- **Backend:** Firebase Authentication (email + anonymous) + Cloud Firestore
- **Local persistence:** AsyncStorage / MMKV for offline pet state and fast reads
- **Usage tracking:** Android UsageStatsManager (requires PACKAGE_USAGE_STATS permission)
- **Drag & Drop:** react-native-gesture-handler + react-native-reanimated
- **Pet carousel:** react-native-reanimated-carousel (or equivalent)
- **Date / time:** dayjs
- **Notifications:** expo-notifications (local)

Why Android-First

The developer has no iOS device. Accurate per-app usage tracking is straightforward on Android via UsageStatsManager. The codebase remains pure React Native so an iOS version can be added later if needed.

Core Firestore Schemas (NoSQL)

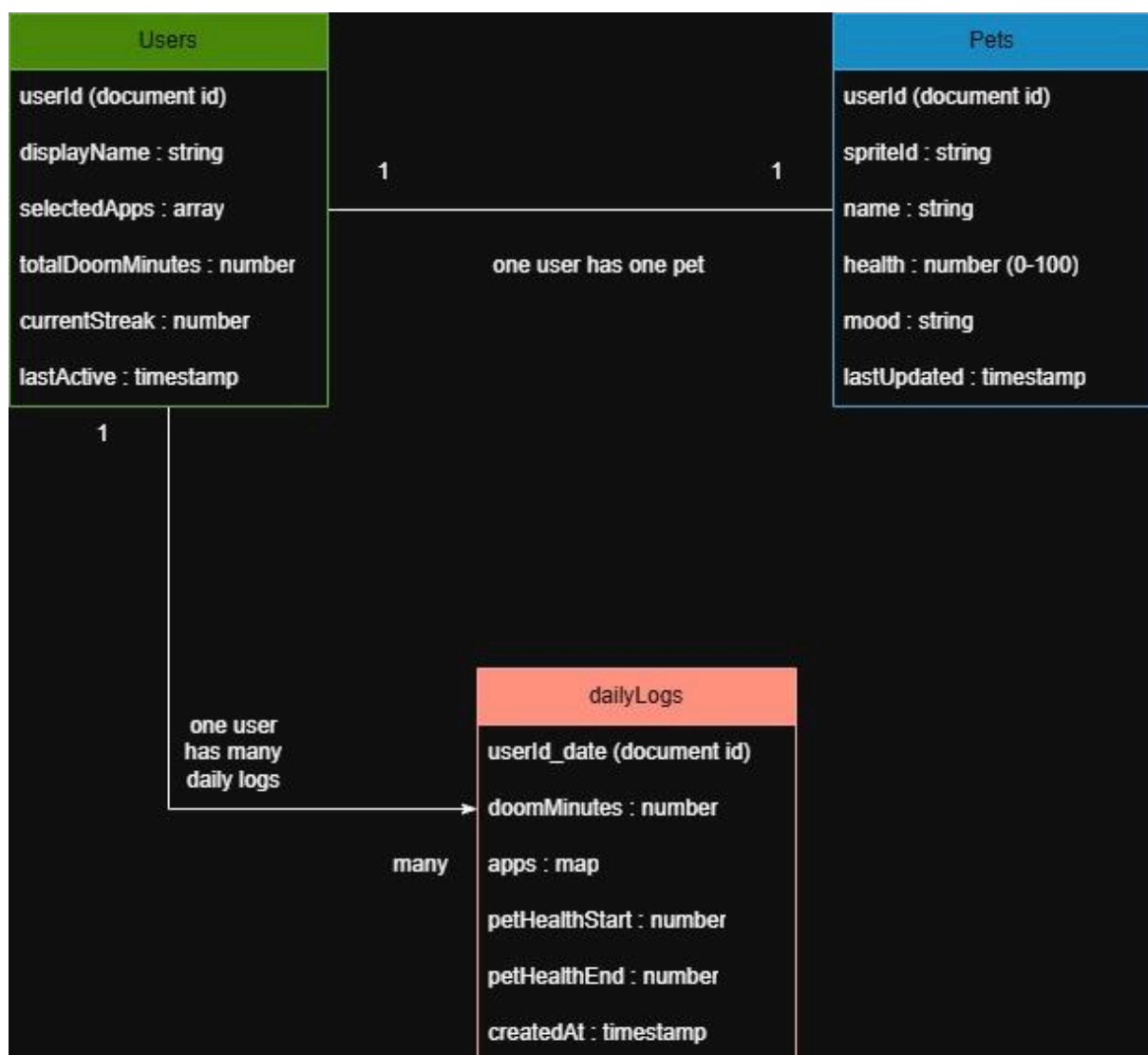
Firestore is a document/collection database. The three collections below store everything the app needs to enforce the Break a Habit theme (doomscroll minutes → pet health) while staying small and queryable.

```
users/{userId}
displayName: string
createdAt: timestamp
selectedApps: array<string> // e.g. ["com.instagram.android"]
totalDoomMinutes: number
currentStreak: number
lastActive: timestamp
pets/{userId}
spriteId: string // "pet1" | "pet2" | "pet3"
name: string
health: number // 0–100
mood: string // "happy" | "tired" | "sick" | "critical"
```

lastUpdated: timestamp
 dailyLogs/{userId}_{yyyy-MM-dd}
 date: string
 doomMinutes: number
 apps: map // { "com.instagram.android": 47, ... }
 petHealthStart: number
 petHealthEnd: number
 createdAt: timestamp

Data Structure & Relationships Diagram

Visual overview of the three collections and their relationships:



How the data supports the Break a Habit theme: Every day a dailyLog is written with the exact minutes spent in the

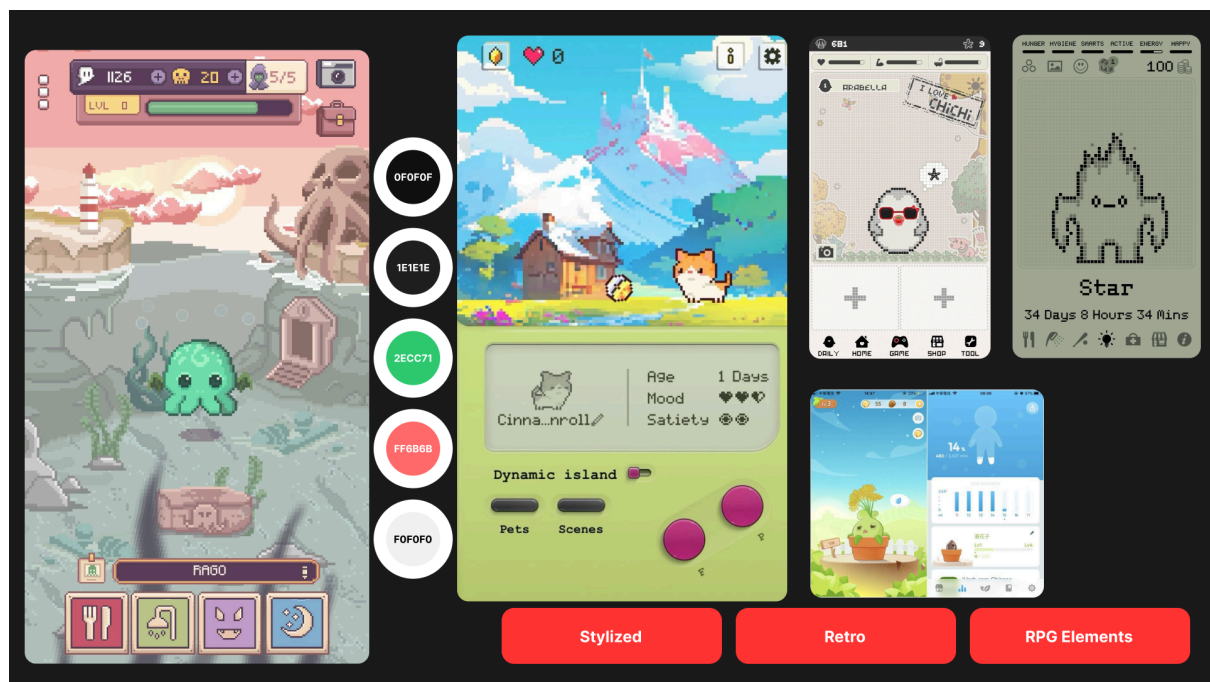
user's selected social apps. That number is used to decrease (or increase) the single pets.health value. The user sees the result immediately on the main screen. This closed loop is what turns abstract "scroll less" advice into a visible, emotional consequence.

Technical Challenges & Approach

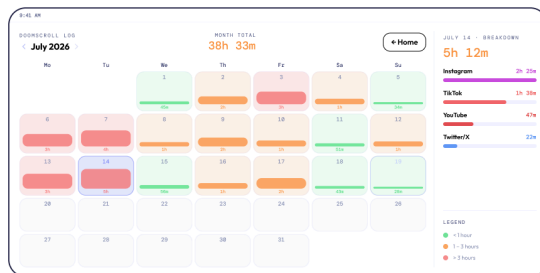
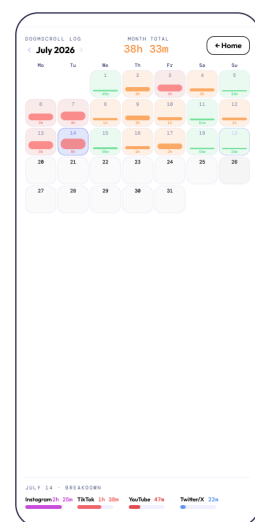
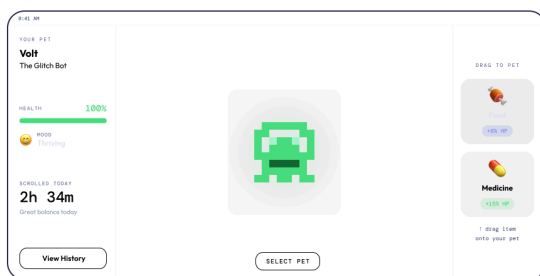
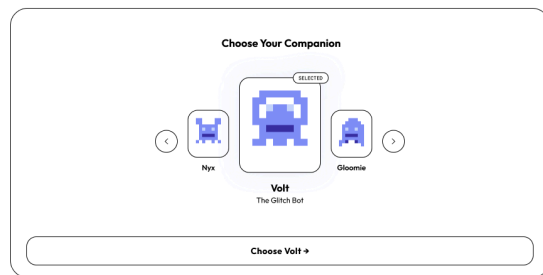
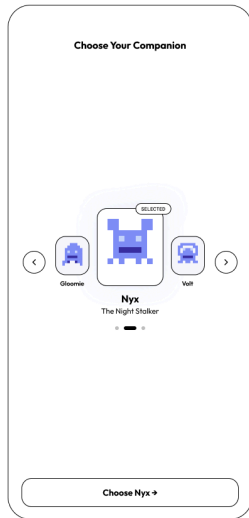
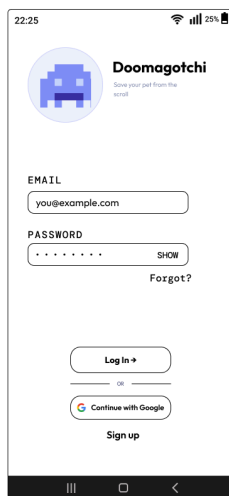
- 1. Usage tracking reliability** — UsageStatsManager requires the special "Usage Access" permission. On first launch the app detects if the permission is missing and opens the system settings page with a clear explanation. Health is recalculated on every app open and while the app is in the foreground.
- 2. Drag & Drop feel in landscape** — react-native-gesture-handler + Reanimated are used so dragging food/medicine onto the pet feels smooth and responsive even on mid-range Android devices.
- 3. Landscape constraint** — All layouts are built with flexbox and percentage-based sizing. Pet Home and History are designed landscape-first and will be tested on multiple aspect ratios (including the A56's 19.5:9).
- 4. Offline behaviour** — Pet health and the latest daily log are cached locally (MMKV). When the device is offline the app still shows the last known state and queues the next write until connectivity returns.
- 5. Permission & privacy** — Only the packages the user explicitly selects are tracked. No other app usage data is read or stored

E) Wireframes & UI/UX Considerations

Visual Direction



The application follows a fun and stylized direction, combining soft pixel-art characters with clean, modern interface elements. The overall mood remains relaxed and playful rather than serious or demanding. All screens prioritise landscape orientation, with the pet as the central focus.



Landscape & Usability Notes

- All primary screens are laid out for landscape (wide pet area, side or bottom controls).
- Drag targets (food, medicine) are large enough for comfortable use in landscape.
- High-contrast health colours so status is readable at a glance.
- System font scaling and basic TalkBack labels supported.

F) Project Roadmap

Phased plan for the rest of the semester.

Content launch tracker

Phase	Focus	Deliverable
Week 1–2	Foundation	Expo + Firebase Auth, basic navigation, landscape layout shell, 3 pixel sprites, swipe carousel
Week 3–4	Core Loop	UsageStatsManager integration, health decay/regen, drag food/medicine onto pet
Week 5–6	History & Polish	Monthly calendar screen, daily logs in Firestore, notifications, idle animations
Week 7–8	Testing	Real-device Android testing, permission flow, landscape on different aspect ratios, bug fixes
Week 8	Submission	Final build, store listing assets, short demo video, documentation

G) Conclusion

Doomagotchi turns the abstract goal of “scroll less” into something you can see and feel. A pixel pet that gets sicker the longer you stay on Instagram gives real emotional weight to every extra minute. The concept fully respects the three locked cards (Break a Habit, Drag & Drop, Landscape Support), stays technically realistic for an Android-first React Native + Firebase student project, and is small enough to finish well. It is focused, achievable, and different enough from generic habit trackers to be worth putting on the Play Store.